Übungsblatt 3: Fortgeschrittene Interrupts und Timer

Mikrocomputertechnik 3 (MCT3)

Zielsetzung

Sie vertiefen Vorlesung 3: Direct vs. Vectored Mode (`mtvec`), CLINT/PLIC, Nested Interrupts/`mepc`-Clobbering sowie die Programmierung eines periodischen System-Ticks über `mtime`/`mtimecmp`.

Bearbeitungszeit: ca. 45–60 Min. | **Orientierungspunkte:** Aufgabe 1: 12 P, Aufgabe 2: 10 P (Gesamt: 22 P)

Teil 1: Theorie — Interrupt-Architektur auf SoCs

Aufgabe 1.1: Direct vs. Vectored Mode

Ein RISC-V-SoC empfängt Interrupts von verschiedenen Quellen. In `mtvec` steht die Basisadresse (`BASE`); das `MODE`-Feld in den untersten Bits steuert das Einsprungverhalten.

1. Erklären Sie den Ablauf im **Direct Mode** ($MODE = 0$). Warum stößt dieser Modus bei vielen Peripheriequellen an Grenzen (Stichwort: Suchlatenz)?
2. Erklären Sie den Ablauf im **Vectored Mode** ($MODE = 1$). Wie berechnet die Hardware die Einsprungadresse für Exception-Code 7 (Machine Timer Interrupt)?

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 1.2: CLINT vs. PLIC

Interrupt-Verwaltung auf RISC-V-SoCs: Core-Local Interruptor (CLINT) und Platform-Level Interrupt Controller (PLIC).

1. Nennen Sie die Hauptaufgaben des CLINT und erklären Sie, warum er „Core Local“ ist (Vorteil gegenüber dem PLIC).
2. Nennen Sie die Hauptaufgaben des PLIC.
3. Der PLIC nutzt einen Zwei-Stufen-Handshake. Erklären Sie **Claim** und **Complete**. Warum ist das Auslesen der Claim-ID auf Hardware-Ebene atomar?

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 1.3: Nested Interrupts und Clobbering

Beim Trap-Eintritt setzt die Hardware typisch `MIE` in `mstatus` auf 0 — weitere Interrupts sind global gesperrt.

1. Warum ist diese „Taubheit“ bei langen ISRs (z. B. langsame I2C-Sequenz) in Echtzeitsystemen problematisch?
2. Bei Nested Interrupts setzt Software `MIE` in der ISR wieder auf 1. Erklären Sie das Clobbering-Problem bezüglich `mepc`.
3. Was muss Software tun, bevor `MIE` wieder auf 1 gesetzt wird?

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Teil 2: Praxis — System-Ticks programmieren

Spezifikation (didaktisches Beispiel, Adressen entsprechen dem verbreiteten SiFive-CLINT-Layout):

- CLINT-Oszillator: 10 MHz (10 000 000 Hz)
- `mtime` an `0x0200BFF8` (64 Bit)
- `mtimecmp` an `0x02004000` (64 Bit)

Aufgabe 2.1: Mathematik des Timers

Berechnen Sie die Anzahl der Oszillator-Takte für exakt 1 ms.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 2.2: Initialisierung in C

Vervollständigen Sie das Snippet, sodass der erste Alarm in 1 ms ausgelöst wird.

```c
#include <stdint.h>

#define CLINT_MTIMECMP 0x02004000u
#define CLINT_MTIME    0x0200BFF8u
#define TICKS_PER_MS   /* TODO: Ergebnis aus 2.1 */

void init_systick(void) {
    /* 1. aktuelle Zeit (mtime) lesen */
    /* TODO */

    /* 2. Intervall addieren, Ziel nach mtimecmp schreiben */
    /* TODO */

    /* 3. Freigabe (Pseudo-Code) */
    /* set_csr(mie, (1u << 7)); */ /* MTIE */
    /* set_csr(mstatus, 8); */     /* MIE */
}
```

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 2.3: Timer-ISR und Quittieren

Der Timer-IRQ bleibt aktiv, solange `mtime >= mtimecmp`.

1. Warum feuert der Interrupt nach `mret` sofort erneut, wenn die ISR nur eine Variable erhöht und zurückkehrt?
2. Schreiben Sie `void timer_isr(void)`, die `uptime_ms` inkrementiert und den nächsten Alarm in 1 ms plant.
3. Warum sollte `__attribute__((interrupt))` (bzw. ein gleichwertiger ABI-/Prolog) verwendet werden?

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)